

**PRAKTIKUM  
PEMROGRAMAN BERORIENTASI OBJEK**

|             |                                |                  |                          |
|-------------|--------------------------------|------------------|--------------------------|
| <b>Nama</b> | <b>Muhammad Rafif Batubara</b> | <b>No. Modul</b> | <b>10</b>                |
| <b>NPM</b>  | <b>2406436253</b>              | <b>Tipe</b>      | <b>Tugas Pendahuluan</b> |

1. State pattern adalah salah satu behavioral design pattern yang memungkinkan sebuah objek untuk mengubah perilakunya secara dinamis saat state internalnya berubah. Ketika pola ini diterapkan, objek tersebut terlihat seperti berganti class. Ide utamanya itu menciptakan class baru buat setiap state yang mungkin dimiliki objek, kemudian memindahkan semua logika spesifik untuk state tersebut ke dalam class barunya. Objek utama (context) menyimpan referensi ke salah satu dari objek state ini dan mendelegasikan semua pekerjaan yang bergantung pada state yang bergantung pada state ke objek tersebut. Transisi dari satu state ke state lain dilakukan dengan mengganti objek state yang disimpan oleh context.

Tujuan utama dari state pattern adalah mengelola state dan transisi state sebuah objek secara bersih. Mengurangi penggunaan if-else-if atau switch case yang besar dan kompleks untuk mengubah perilaku objek berdasarkan state-nya saat ini. Membuat kode di dalam context menjadi jauh lebih sederhana atau bersih dan perubahan state menjadi eksplisit dengan memisahkan logika state ke dalam class terpisah

Manfaat :

- Menghindari kondisional yang rumit (if-else-if atau switch case) agar kode lebih sederhana dan mudah dibaca
- Mengikuti single responsibility principle, dengan memisahkan logika spesifik untuk satu state ke dalam classnya sendiri dapat meningkatkan kohesi
- Mengikuti open/close principle sehingga penambahan state menjadi mudah di masa depan
- Meningkatkan maintainability karena kode menjadi lebih fleksibel, robust, dan lebih mudah dipelihara
- Transisi jelas karena transisi antar state menjadi eksplisit

**Referensi:**

- GeeksforGeeks, "State Design Pattern," *GeeksforGeeks*, Aug. 05, 2017.  
<https://www.geeksforgeeks.org/system-design/state-design-pattern/> (accessed 16 November

2025, 18.30WIB)

- “State in Java / Design Patterns,” *Refactoring.guru*, 2024.  
<https://refactoring.guru/design-patterns/state/java/example> (accessed 16 November 2025, 18.31WIB)
- “State Design Pattern in Java | DigitalOcean,” *www.digitalocean.com*.  
<https://www.digitalocean.com/community/tutorials/state-design-pattern-java> (accessed 16 November 2025, 18.32WIB)

2. Merupakan behavioral design pattern yang memungkinkan kita untuk mendefinisikan sebuah strategi, menempatkan masing-masing algoritma tersebut ke dalam kelas yang terpisah dan objek-objek tersebut dapat saling ditukarkan. Objek context menyimpan referensi ke salah satu objek strategy dan mendelegasikan pekerjaan kepadanya

Perubahan algoritma/perilaku ketika runtime dicapai dengan 3 konsep utama yaitu :

- Enkapsulasi : pattern ini dimulai dengan mengidentifikasi sekelompok algoritma yang terkait kemudian di enkapsulasi di dalam classnya sendiri (class strategi ini dibuat untuk mengimplementasikan interface yang sama)
- Komposisi dan delegasi context memiliki sebuah referensi ke salah satu objek strategi yang sesuai dengan interface tadi. Jadi ketika context perlu menjalankan algoritma ia mendelegasikan pemanggilan tersebut ke objek strategi yang saat ini dipegangnya (tidak melakukannya sendiri)
- Karena context hanya memegang referesni ke sebuah interface strategi, referensi tersebut dapat diubah kapan saja selama runtime. Context menyediakan setter yang memungkinkan client untuk mengganti objek strategi yang dipegangnya dengan objek strategi lain

Ketika referensi strategi itu diganti, perilaku context langsung berubah pada pemanggilan berikutnya, karena ia kini mendelegasikan pekerjaan ke objek yang berbeda memungkinkan pemilihan algoritma terjadi saat runtime

#### Referensi:

- M. Gift, “Understanding the Strategy Pattern,” *Medium*, Jul. 10, 2025.  
<https://medium.com/@infinitemagnus/understanding-the-strategy-pattern-2b7bb5b302ee>  
(accessed Nov. 16, 2025, 18.40WIB).

- GeeksforGeeks, “Strategy Design Pattern,” *GeeksforGeeks*, Apr. 29, 2016.  
<https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/> (accessed Nov. 16, 2025, 18.42WIB).
- “Design Patterns: Strategy in Java,” *Refactoring.guru*, 2014.  
<https://refactoring.guru/design-patterns/strategy/java/example> (accessed Nov. 16, 2025, 18.43WIB).

### 3. Perbedaan intent :

- State pattern : mengelola state sebuah objek. Memungkinkan sebuah objek mengubah perilakunya secara dinamis ketika state internalnya berubah. Fokus terhadap “apa” sebuah objek tersebut
- Strategy pattern : mendefinisikan keluarga algoritma dan membuatnya dapat saling dipertukarkan (interchangeable). Memungkinkan client memilih algoritma mana yang akan digunakan saat runtime. Berfokus pada “bagaimana” sebuah objek melakukan sesuatu

### Perbedaan struktur :

- State Pattern : objek state yang konkret bertanggung jawab atas transisi state. objek state perlu menyimpan referensi kembali ke context agar dapat mengubah state dari context. perubahan state diatur secara internal. Context mendelegasikan perilakunya ke state saat ini dan state tersebut lah yang memutuskan kapan harus beralih ke state berikutnya
- Strategy pattern : objek strategy merupakan algoritma yang terisolasi dan mereka tidak bertanggung jawab untuk mengganti strategi. Tidak ada referensi ke context karena objek strategi tidak menyadari adanya context. Mereka adalah algoritma mandiri yang hanya mengeksekusi tugas dan mengembalikan nilai. Perubahan strategi selalu diatur secara eksternal oleh client (client memutuskan strategi mana yang harus digunakan context).

### Referensi:

- GeeksforGeeks, “Difference Between State and Strategy Design Pattern in Java,” *GeeksforGeeks*, Feb. 19, 2021.  
<https://www.geeksforgeeks.org/java/difference-between-state-and-strategy-design-pattern-in-j>

[ava/](#) (accessed Nov. 16, 2025, 18.57WIB).

- iamprovidence, “Strategy vs State pattern,” *Medium*, Feb. 04, 2023.  
<https://medium.com/@iamprovidence/strategy-vs-state-pattern-6168cc102c91> (accessed Nov. 16, 2025, 18.57WIB).
- E. Kovko, “Baeldung,” *Baeldung on Computer Science*, Jan. 03, 2023.  
<https://www.baeldung.com/cs/design-state-pattern-vs-strategy-pattern> (accessed Nov. 16, 2025, 19.00WIB).

4. State pattern sering digunakan untuk mengelola GameState karena menyediakan cara yang bersih untuk mengimplementasikan state machine, memungkinkan sebuah objek untuk mengubah perilakunya secara dinamis ketika state internalnya berubah.

Kelebihan dibandingkan dengan if-else atau switch case :

- Mengurangi kompleksitas
- Mengikut single responsibility principle (meningkatkan cohesion)
- Mengikut open/closed principle. Sangat mudah untuk menambahkan state baru di masa depan
- Manajemen transisi lebih aman dan jelas. Setiap class state mengelola transisinya sendiri, pattern diimplementasikan dengan method enter() dan exit(). Memastikan logika dieksekusi secara konsisten setiap kali state berganti (mencegah bug transisi)

#### Referensi:

- GeeksforGeeks, “State Design Pattern,” *GeeksforGeeks*, Aug. 05, 2017.  
<https://www.geeksforgeeks.org/system-design/state-design-pattern/> (accessed Nov. 16, 2025, 19.10WIB).
- R. Nystrom, “State · Design Patterns Revisited · Game Programming Patterns,” *Gameprogrammingpatterns.com*, 2009. <https://gameprogrammingpatterns.com/state.html> (accessed Nov. 16, 2025, 19.10WIB).
- M. Gorin, “Stop Writing If-Else Trees: Use the State Pattern Instead,” *Medium*, Apr. 10, 2025.

<https://maxim-gorin.medium.com/stop-writing-if-else-trees-use-the-state-pattern-instead-1fe9ff39a39c> (accessed Nov. 16, 2025, 19.11 WIB).

5. Strategy pattern ideal ketika kita ingin algoritma-algoritma dapat saling dipertukarkan, dipilih atau diubah pada saat runtime. Sering muncul dalam kode yang memiliki method dengan banyak if-else atau switch case statement digunakan untuk memilih perilaku yang berbeda berdasarkan suatu kondisi. Contoh skenario adalah pembayaran E-commerce. Misal ShoppingCart sebagai context. Class perlu memproses total pembayaran, pelanggan dapat memilih berbagai cara untuk membayar. ShoppingCart tidak menggunakan method (checkout()) besar dengan if-else statement dan menggunakan strategy pattern dimana terdapat strategy interface (PaymentStrategy). concrete strategy (class terpisah untuk setiap metode pembayaran yang mengimplementasikan interface seperti CreditCartStrategy, PaypalStrategy, QrisStrategy, dll). ShoppingCart memiliki referensi ke PaymentStrategy, ketika checkout client memilih metode pembayaran kemudian aplikasi mengatur strategi ShoppingCart ke metode yang dipilih. Saat ShoppingCart memanggil checkout(), ia mendelegasikan strategi yang dipegangnya tanpa tahu cara kerjanya

#### Referensi:

- GeeksforGeeks, "Strategy Design Pattern," *GeeksforGeeks*, Apr. 29, 2016.  
<https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/> (accessed Nov. 16, 2025, 19.20 WIB).
- S. Team, "Stackify," *Stackify*, Feb. 26, 2025.  
<https://stackify.com/strategy-pattern-definition-examples-and-best-practices/> (accessed Nov. 16, 2025, 19.21 WIB).
- iluwatar, "Strategy Pattern in Java: Streamlining Object Behaviors with Interchangeable Algorithms," *Java Design Patterns*, 2025. <https://java-design-patterns.com/patterns/strategy/> (accessed Nov. 16, 2025, 19.25 WIB).

6. Context adalah kelas utama yang perilakunya ingin kita ubah berdasarkan state internalnya. Peran utamanya adalah menyimpan referensi ke salah satu objek state konkret yang mewakili state-nya saat ini. State mendefinisikan perilaku yang terkait dengan kondisi tertentu. State memiliki 2 peran yaitu

state interface yang mendefinisikan kontrak yang harus dimiliki oleh semua state konkret dan concrete class yang merupakan implementasi spesifik dari state interface (setiap class mengenkapsulasi logika dan perilaku yang berhubungan untuk satu state tertentu dari context).

Interaksi kedua hal tersebut terjadi melalui delegasi. Ketika client memanggil sebuah metode pada context, context mendelegasikan panggilan tersebut ke objek concrete state yang saat ini dipegangnya. Objek state kemudian mengeksekusi perilakunya yang spesifik. Transisi state terjadi ketika context dapat menyediakan method yang memungkinkan objek state itu sendiri untuk memicu transisi sehingga memindahkan context ke state yang baru.

### Referensi:

- GeeksforGeeks, "State Design Pattern in Java," *GeeksforGeeks*, Jun. 28, 2024.  
<https://www.geeksforgeeks.org/system-design/state-design-pattern-in-java/> (accessed Nov. 16, 2025, 19.30WIB).
- "State Design Pattern in Java | DigitalOcean," *www.digitalocean.com*.  
<https://www.digitalocean.com/community/tutorials/state-design-pattern-java> (accessed Nov. 16, 2025, 19.34WIB).
- Mohamed-Ali, "The State Design Pattern: A Comprehensive Guide," *Medium*, Apr. 21, 2025.  
<https://medium.com/@moali314/the-state-design-pattern-a-comprehensive-guide-daa96ac7dbc7> (accessed Nov. 16, 2025, 19.35WIB).

7. Strategy pattern memisahkan class utama (context) dari algoritma/strategi yang digunakannya. Context diprogram untuk bekerja melawan interface strategi bukan implementasi strategi yang konkret. Cara untuk menambahkan strategi baru tanpa modifikasi adalah sebagai berikut :
  - Buat kelas strategi baru
  - Implementasikan interface strategi. Class baru harus menyediakan implementasi konkret untuk metode yang didefinisikan di dalam interface sebelumnya
  - Gunakan strategi baru saat runtime. Context bergantung pada interface dan langsung dapat menggunakan strategi baru. Klien sekarang dapat membuat instansi dari strategi baru dan meneruskannya ke context kapan saja, mengubah perilakunya secara dinamis tanpa mengubah kode di dalam context itu sendiri

### Referensi:

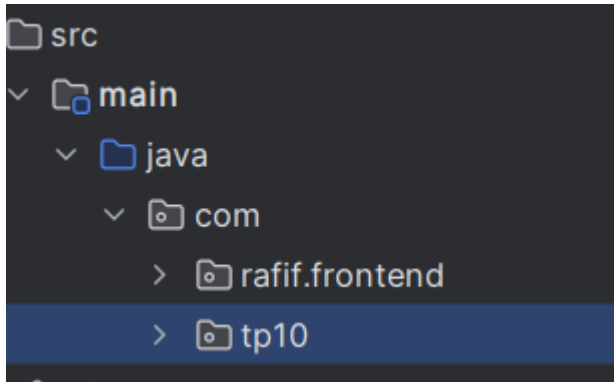
- Ioan Tinca, "The Open/Closed Principle and Strategy Pattern," *dzone.com*, Apr. 02, 2018. <https://dzone.com/articles/the-open-closed-principle> (accessed Nov. 16, 2025, 19.44WIB).
- GeeksforGeeks, "Strategy Design Pattern," *GeeksforGeeks*, Apr. 29, 2016. <https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/> (accessed Nov. 16, 2025, 19.44WIB)
- Ioan Tinca, "The Open/Closed Principle and Strategy Pattern," *dzone.com*, Apr. 02, 2018. <https://dzone.com/articles/the-open-closed-principle> (accessed Nov. 16, 2025, 19.44WIB)

8. Mini project yang saya buat adalah 1v1 combat menggunakan `ShapeRenderer` untuk visualisasi dimana player yang kita kontrol dan AI sebagai musuh memiliki bentuk `rectangle`. State pattern diimplementasikan menjadi state dari setiap karakter (`Fighter`) bertindak sebagai context dan memegang satu state aktif seperti `StandState`, `WalkState`, `PunchState`, `JumpState`, `BlockState`, `HitState`. Semua logika yang ada (dari diinput di ingame sampai durasi serangannya habis) didelegasikan ke objek state aktif agar class `fighter` utama tidak terlalu kompleks oleh `if-else` statement. Strategy pattern implementasinya ada untuk mengatur logika dari AI (musuh). Karakter AI memiliki objek strategy seperti `AggressiveStrategy` dan `DefensiveStrategy`. Tanpa memproses input keyboard, karakter AI bakal memanggil metode `execute()` dari interface `FightingStrategy` yang memberikan perintah untuk AI mengejar, menyerang (`PerformPunch`) atau bertahan (`PerformBlock`), sehingga gaya bertarung AI menjadi mudah diganti.

9. Link demo mini project

<https://youtu.be/m08mS3o14Wg>

\*project tp10 dtaruh di package baru disebelah com.rafi.f.frontend



LWJGL3Launcher juga sudah di set untuk run mini project tp10

```
private static Lwjgl3Application createApplication() { 1 usage  👤 Rafif1901 *  
    return new Lwjgl3Application(new com.tp10.Main(), getDefaultConfiguration());  
}
```